

Calibration de modèles EDO d'activité mitochondriale par algorithme génétique

Tuteur :
Mr.Leguebe

Auteur :
Dama Diatta
Gonçalves de Matos Léa
Repauzet Olivier
Sidi Mohamed
Sores Kélian

Licence 3 Ingénierie Mathématique
Université de Bordeaux
2025-2026

Sommaire

Résumé	1
1 Introduction	2
2 L’Article de Bertram	2
2.1 Contexte du modèle	2
2.2 Explication biologique de la synthèse de l’ATP	3
3 Formulation mathématique du modèle	3
3.1 Variables d’état et structure du système	3
3.2 Équations de flux	3
3.3 Mise sous forme de problème de Cauchy	4
3.4 Existence et unicité d’une solution locale	4
3.5 Les flux intervenant dans le modèle	4
4 Implémentation	6
4.1 Reprise du modèle à partir de CellML	6
4.2 Reconstruction et exploitation en Python	6
4.3 Développement, débogage et validation en C++	6
5 Le modèle de Bertram étendu	8
5.1 Modélisation des expériences	8
5.2 Modification et extension de l’implémentation	9
5.3 Normalisation de l’oxygène	9
6 Calibration du modèle	10
6.1 Régression linéaire	11
6.2 Construction de la calibration	11
6.3 Principe de la calibration	12
6.4 Algorithme génétique	12
7 Perspectives et Conclusion	14

Résumé

Ce travail présente l’étude, l’implémentation numérique et la calibration d’un modèle mathématique décrivant la dynamique mitochondriale. Fondé sur les travaux de *Bertram et al.* (2006), le système d’équations différentielles ordinaires (EDO) modélise les flux biochimiques essentiels à la production d’ATP et à la régulation calcique.

Après une phase de transcription et de validation numérique sous Python, un programme en C++, utilisant un solveur de Runge-Kutta d’ordre 4 pour garantir la stabilité du problème de Cauchy associé. Le modèle initial a été étendu pour intégrer des dynamiques d’ajouts d’ADP conformes aux protocoles expérimentaux fournis. Enfin, nous avons utilisé un algorithme génétique, afin de reproduire au plus proche les résultats expérimentaux.

1 Introduction

Les mitochondries sont des organites essentiels au fonctionnement des cellules eucaryotes, assurant la production principale d'énergie sous forme d'ATP par le processus de phosphorylation oxydative. Leurs activités dépendent fortement de l'environnement de la cellule, notamment de la disponibilité en substrats énergétiques (comme le glucose) et en ions calcium Ca^{2+} , qui jouent un rôle clé dans la régulation des flux métaboliques.

La dynamique mitochondriale peut être décrite à l'aide de modèles mathématiques traduisant les mécanismes biochimiques en systèmes d'équations différentielles ordinaires. Ces modèles permettent de relier les processus microscopiques (flux ioniques, réactions enzymatiques, transport membranaire) à des observables macroscopiques mesurables expérimentalement, comme la consommation d'oxygène ou la production d'ATP.

Il existe de nombreux modèles mitochondriaux dans la littérature, souvent développés par des biologistes et intégrant un grand nombre de mécanismes couplés. Toutefois, leur complexité rend parfois difficile leur analyse mathématique et leur implémentation numérique. Dans ce projet, nous nous appuyons sur le modèle proposé par *Richard Bertram et al.* (2006), qui introduit plusieurs simplifications structurales tout en conservant les mécanismes essentiels de la réponse mitochondriale à l'ajout de substrat et de calcium.

Ce modèle décrit notamment l'évolution temporelle des concentrations en calcium Ca^{2+} et en adénosine triphosphate (ATP), ainsi que les flux associés au transport et aux réactions métaboliques internes. Il fournit ainsi un cadre permettant d'étudier la production d'énergie en fonction des conditions expérimentales.

L'objectif de ce projet est : reconstruire mathématiquement le modèle sous forme de problème de Cauchy, en assurer l'implémentation numérique fidèle aux données de l'article, puis procéder à la calibration paramétrique à partir de données expérimentales de consommation d'oxygène. Cette démarche s'inscrit dans une approche pluridisciplinaire mobilisant des connaissances de biologie, de physique, d'analyse mathématique, de programmation scientifique et d'optimisation numérique.

2 L'Article de Bertram

2.1 Contexte du modèle

Dans l'article suivant *Bertram et al.* [1], les auteurs proposent une reformulation simplifiée du modèle de Magnus et Keizer, initialement développé pour décrire de manière détaillée les mécanismes de production d'ATP au sein de la mitochondrie.

Le modèle original de Magnus–Keizer constitue une description biophysique fine de la phosphorylation oxydative, intégrant un grand nombre de variables, de flux ioniques et de réactions enzymatiques couplées. Bien que très précis, ce modèle présente une complexité importante rendant son analyse mathématique, sa simulation numérique et sa calibration particulièrement délicates.

Partant de ce constat, Bertram et al. introduisent un modèle simplifié visant à reproduire qualitativement et quantitativement les comportements essentiels du modèle complet, tout en réduisant significativement sa dimension et le nombre de paramètres.

Ce modèle simplifié permet notamment d'étudier la réponse mitochondriale aux variations du flux glycolytique (représentant l'apport en glucose) et aux variations de la concentration en calcium intracellulaire. Il fournit ainsi un cadre dynamique pour analyser l'évolution de grandeurs observables telles que la concentration en NADH, la consommation d'oxygène ou encore le potentiel de membrane mitochondriale.

Dans la suite de ce travail, nous nous appuyons sur cette version simplifiée afin d'en reconstruire la formulation mathématique, d'en assurer l'implémentation numérique, puis d'en étudier la calibration à partir de données expérimentales.

2.2 Explication biologique de la synthèse de l'ATP

La mitochondrie est l'organe responsable de la production d'énergie cellulaire par phosphorylation oxydative. Ce processus repose sur une chaîne de transport d'électrons localisée dans la membrane interne mitochondriale (la mitochondrie possède une caractéristique propre, qui est une structure à double membrane), où l'oxydation des substrats énergétiques conduit à la formation d'un gradient de protons.

Ce gradient est ensuite utilisé par l'ATP synthase pour produire de l'adénosine triphosphate (ATP), principale source d'énergie utilisable par la cellule. La dynamique de ce système dépend fortement de plusieurs variables intracellulaires, notamment la concentration en NADH, en ions calcium Ca^{2+} , du potentiel de membrane $\Delta\psi$, ainsi que les concentrations en ADP_m .

Le calcium joue un rôle régulateur majeur : son entrée dans la mitochondrie stimule certaines enzymes du cycle de Krebs, augmentant ainsi la production de NADH et, donc, la synthèse d'ATP.

L'ajout de substrat énergétique ou une augmentation du calcium modifie donc l'état dynamique du système mitochondrial.

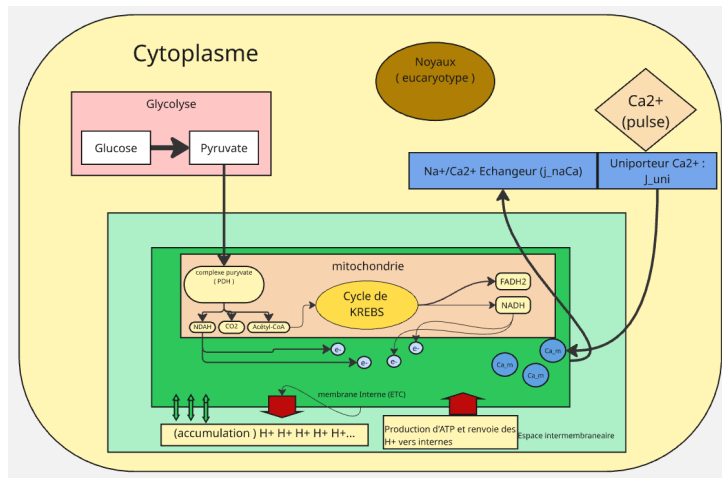


Figure 1: Représentation des échanges de flux mitochondriaux

3 Formulation mathématique du modèle

3.1 Variables d'état et structure du système

Le modèle simplifié proposé par Bertram et al. repose sur une description dynamique d'un ensemble de flux qui sont réduits à des variables représentant les grandeurs essentielles du fonctionnement mitochondrial.

On considère un vecteur d'état

$$X(t) = (NADH(t), C_m(t), \text{ADP}_m(t), \Delta\psi(t)),$$

L'évolution temporelle de chaque variable d'état est déterminée par un bilan de flux, représentant les mécanismes biochimiques en jeu.

3.2 Équations de flux

Chaque variation temporelle est décrite comme la différence entre des flux entrants et sortants. De manière générale, si J_i désigne un flux associé à un mécanisme biologique donné, l'évolution d'une variable d'état s'écrit comme une combinaison de ces flux.

On obtient ainsi un système différentiel non linéaire de la forme :

$$\begin{cases} \frac{dNADH_m}{dt} = \gamma(J_{PDH} - J_o) \\ \frac{dCa_m}{dt} = f_m(J_{uni} - J_{NaCa}) \\ \frac{d\Delta\Psi}{dt} = \frac{1}{C_m} (J_{Hres} - J_{Hatp} - J_{Hleak} - J_{ANT} - J_{NaCa} - 2J_{uni}) \\ \frac{dADP_m}{dt} = \gamma(J_{ANT} - J_{F1F0}) \end{cases} \quad (1)$$

3.3 Mise sous forme de problème de Cauchy

Le système précédent peut être écrit de manière compacte sous la forme :

$$\frac{dX}{dt} = F(X, \theta), \quad X(0) = X_0,$$

où :

- $X(t) \in R^n$ est le vecteur d'état,
- θ désigne l'ensemble des paramètres biologiques du modèle,
- $F : R^n \rightarrow R^n$ est une fonction non linéaire regroupant l'ensemble des flux.

Il s'agit donc d'un problème de Cauchy associé à un système différentiel autonome non linéaire. La non-linéarité provient principalement des termes tels que J_{PDH} que l'on va expliquer afin de finir la compréhension globale du modèle, mais avant de démontrer l'existence et l'unicité locales de la solution.

3.4 Existence et unicité d'une solution locale

Les fonctions intervenant dans les flux sont continues, avec des exponentielles, des polynômes, et des fractions rationnelles donc F est de classe C^1 donc localement lipschitziennes par rapport aux variables d'état.

Par conséquent, d'après le théorème de Cauchy–Lipschitz, pour toute condition initiale (t_0, y_0) , il existe une unique solution locale

$$y : I \rightarrow R^n$$

du problème de Cauchy

$$\begin{cases} y'(t) = f(t, y(t)), \\ y(t_0) = y_0, \end{cases}$$

où I est un intervalle ouvert contenant t_0 .

Cette propriété garantit la cohérence mathématique du modèle et justifie l'étude numérique de sa dynamique.

3.5 Les flux intervenant dans le modèle

Les flux représentent les échanges et réactions au sein de la mitochondrie. On considère ainsi neuf flux principaux : Le premier flux correspond au flux contenant l'inhibition par $NADH$ et l'activation par Ca^{2+} . On le note J_{PDH} (référence au complexe du pyruvate) :

$$J_{PDH} = \left(\frac{p_1}{p_2 + \frac{NADH_m}{NAD_m}} \right) \left(\frac{Ca_m}{p_3 + Ca_m} \right) J_{GPDH}$$

On notera que lorsque $NADH$ augmente, le flux J_{PDH} diminue, tandis que le calcium agit comme un activateur.

Le flux lié à la consommation d'oxygène, noté J_o , est défini par :

$$J_o = \left(\frac{p_4 NADH_m}{p_5 + NADH_m} \right) \left(\frac{1}{1 + \exp\left(\frac{\Delta\Psi - p_6}{p_7}\right)} \right)$$

On distingue deux parties dans cette expression :

- La première fraction correspond au carburant utilisé, avec des paramètres ajustés expérimentalement.
- La seconde fraction représente un frein électrique stabilisant le système.

Le flux de protons respiratoires, noté J_{Hres} , correspond au pompage de protons par la chaîne respiratoire :

$$J_{Hres} = \left(\frac{p_8 NADH_m}{p_9 + NADH_m} \right) \left(\frac{1}{1 + \exp\left(\frac{\Delta\Psi - p_{10}}{p_{11}}\right)} \right) \quad (2)$$

Deux flux similaires (à coefficients près), ajustés pour se rapprocher du modèle de Magnus-Keizer, sont :

$$J_{Hatp} = \left(\frac{p_{13}}{p_{13} + ATP_m} \right) \left(\frac{p_{12}}{1 + \exp\left(\frac{p_{14} - \Delta\Psi}{p_{15}}\right)} \right) \quad (3)$$

$$J_{F1F0} = \left(\frac{p_{13}}{p_{13} + ATP_m} \right) \left(\frac{p_{16}}{1 + \exp\left(\frac{p_{14} - \Delta\Psi}{p_{15}}\right)} \right) \quad (4)$$

On pense à noter la fuite des protons qui est dissipée au cours des échanges par une fonction linéaire, on note J_{Hleak} , qui est définie par:

$$J_{Hleak} = p_{17} \Delta\Psi + p_{18} \quad (5)$$

Il y a un équilibre naturel qui se forme entre ATP et ADP qui est favorisé par $\Delta\Psi$, Ici RAT est le rapport entre l' ATP et l' ADP .

$$RAT_m = \frac{ATP_m}{ADP_m} \quad (6)$$

On obtient la formule suivante :

$$J_{ANT} = p_{19} \left(\frac{RAT_m}{RAT_m + p_{20}} \right) \exp(0.5 FRT \Delta\Psi) \quad (7)$$

On caractérise l'apport en calcium cytosolique (qui vient du cytoplasme) par le flux uniporteur (transporte seulement les ions Ca^{2+}). L'apport va avoir un impact direct sur l'équilibre du potentiel de protons, et donc sur la création de l' ATP , on le définit par :

$$J_{uni} = (p_{21} \Delta\Psi - p_{22}) Ca_c^2 \quad (8)$$

Inversement au flux uniporteur il existe le flux J_{NaCa} qui est un flux sortant de la mitochondrie qui vient diminuer le potentiel de charge interne et ainsi réguler la charge de la mitochondrie. On le définit par :

$$J_{NaCa} = p_{23} \left(\frac{Ca_m}{Ca_c} \right) \exp(p_{24} \Delta\Psi) \quad (9)$$

4 Implémentation

4.1 Reprise du modèle à partir de CellML

Une fois les principes biologiques, physiques et mathématiques du modèle compris, une première étape a consisté à reprendre l'implémentation existante du modèle de Bertram disponible via la ressource *CellML* [2]. CellML est un standard ouvert fondé sur le langage XML, permettant de décrire et de partager des modèles mathématiques.

L'intérêt de cette ressource était de fournir une transcription exploitable du modèle présenté dans l'article et de constituer un point de référence indispensable pour vérifier la cohérence de nos propres implémentations. Cependant, le format CellML reste relativement peu lisible dans un premier temps, en particulier pour une utilisation directe dans un cadre de calcul scientifique classique.

Un travail de lecture, de traduction et de réorganisation du code extrait a donc été nécessaire afin d'identifier clairement les variables d'état, les paramètres, les flux et les équations différentielles associées. Cette étape a permis d'obtenir une première compréhension opérationnelle du modèle, préalable indispensable à sa reconstruction dans des langages plus adaptés à la simulation numérique.

L'objectif de cette première phase n'était pas encore de modifier le modèle, mais bien de garantir une reproduction fidèle de sa structure et de ses sorties de référence.

4.2 Reconstruction et exploitation en Python

À partir du code issu de CellML, la suite a consisté à reconstruire une version exploitable du modèle en Python. Ce langage a été retenu dans un premier temps pour sa robustesse, sa lisibilité et la simplicité de génération de sorties graphiques.

L'essentiel du travail n'a pas porté sur la formulation mathématique elle-même, déjà présente dans la ressource initiale, mais sur la réécriture du code dans une structure plus claire, ainsi que sur l'adaptation des affichages nécessaires à l'analyse des résultats. Il a notamment fallu reformater les sorties, vérifier la cohérence des variables manipulées et reproduire les courbes de référence issues de l'article.

Cette implémentation intermédiaire a permis de confronter les résultats numériques obtenus aux comportements attendus, tout en servant de support de comparaison avec l'implémentation plus avancée développée par la suite en C++.

Python a ainsi constitué un environnement de transition entre la ressource brute extraite de CellML et une version plus malléable du modèle, pensée pour être étendue et calibrée.

4.3 Développement, débogage et validation en C++

Afin de disposer d'un modèle plus modulable et surtout réutilisable et mieux adapté à des simulations plus importantes, nous avons ensuite développé une implémentation complète en C++. L'objectif n'était pas uniquement de reproduire les résultats du modèle de Bertram, mais également de construire une base suffisamment propre pour permettre des extensions ultérieures, notamment l'ajout de nouvelles variables d'état et la calibration paramétrique, qui fait l'objet de la suite de notre projet.

L'architecture retenue repose sur une séparation claire entre les paramètres, les flux, les équations d'évolution et le solveur numérique. Les paramètres du modèle ont été stockés dans un fichier externe, puis chargés dans le programme, ce qui facilite les modifications et les essais de calibration. Les équations de flux et les dérivées des variables d'état ont été regroupées dans une classe dédiée représentant le modèle mitochondrial, la "Class Bertram".

Une part importante du travail a porté sur le débogage du code. En effet, bien que la structure des équations soit directement issue de l'article, plusieurs difficultés sont apparues lors de l'implémentation, en particulier sur la question des unités. Une incohérence d'analyse dimensionnelle dans certains termes conduisait à des sorties incorrectes, voire à des dynamiques incompatibles avec les résultats attendus. La correction de ces erreurs a demandé une vérification détaillée des dimensions physiques de chaque flux,

ainsi qu'un contrôle systématique de leur contribution dans les équations différentielles.

Concernant la résolution numérique, nous avons retenu une méthode de Runge-Kutta d'ordre 4 (RK4). Ce choix permet de conserver une méthode explicite simple à implémenter, tout en améliorant nettement la précision et la stabilité par rapport à un schéma d'Euler implicite. Cette propriété était particulièrement importante dans notre cas, le modèle étant simulé sur des horizons temporels relativement longs avec des pas de temps très petits.

Même si RK4 est explicite, sa région de stabilité est bien meilleure que celle d'un Euler explicite simple. Par exemple pour une équation test de type $y' = \lambda y$, Euler donne $y^{n+1} = (1 + \Delta t \lambda) y^n$, on remarque que cela est instable dès que la fonction devient trop raide (c'est à dire que la fonction explose à de fortes valeurs de $\Delta t \lambda$)

Or, RK4 donne par le même fonctionnement $y^{n+1} = R(\Delta t \lambda) y^n$ avec $R(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24}$, Cette approximation de e^x reproduit bien mieux la dynamique local, donc la solution numérique est plus fiable pour des pas raisonnables.

Remarque 1 Pour vérifier la validité de notre solveur, on a dû le comparer à une équation différentielle dont on connaît la solution et la forme analytique. Pour cela, on prend une fonction simple du type $y' = ty$ avec t grand pour observer la stabilité d'une fonction dite "raide", et on applique un log afin d'avoir une courbe linéaire.

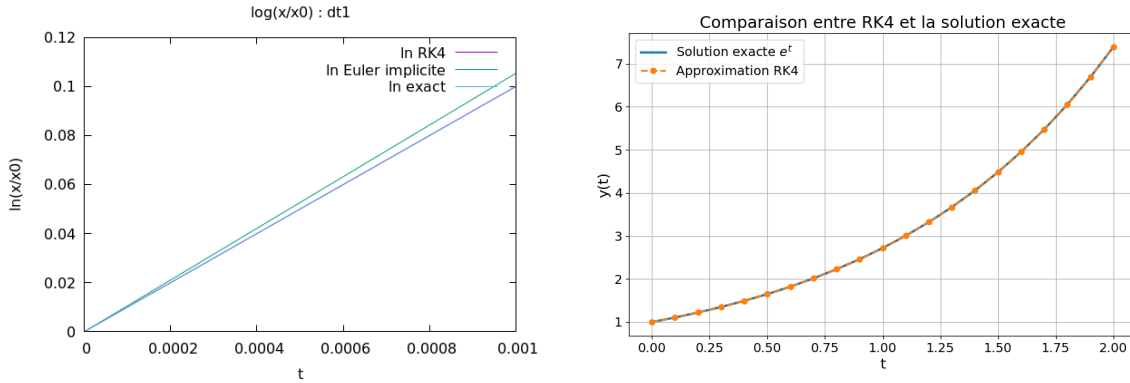
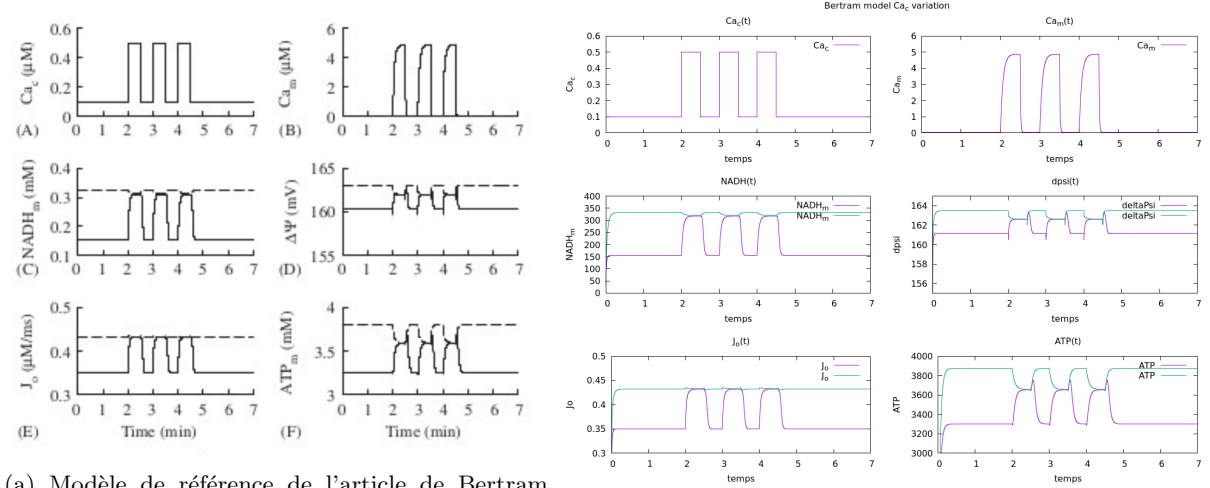


Figure 2: Comparaison de l'approximation en fonction de Δt de RK^4 contre Euler implicite (connu pour sa robustesse) d'une équation $y'(t) = \alpha y(t)$ avec $\alpha \gg 0$, on observe bien une convergence pour des Δt suffisamment petits

Une fois ces étapes de validation franchies, l'implémentation C++ a permis de reproduire les sorties du modèle de Bertram de manière satisfaisante.



(a) Modèle de référence de l'article de Bertram : simulation de la réponse mitochondriale à des ajouts (pulse) de Ca^{2+} (b) Sortie d'implémentation sous les mêmes conditions que le modèle de Bertram.

Figure 3: Comparaison entre les sorties obtenues avec notre implémentation C++ et les figures de référence issues de Bertram.

5 Le modèle de Bertram étendu

5.1 Modélisation des expériences

On est passé à la suite de notre projet; il faut comprendre qu'il y a une différence principale entre les expériences et le modèle de Bertram, c'est-à-dire que de l'ADP est ajouté au bain qui contient les mitochondries, tandis que Bertram et al. supposent que le ratio ADP/ATP est le même à l'intérieur des mitochondries et à l'extérieur.

Afin de caractériser ces nouvelles données, on doit augmenter le nombre de variables de notre modèle et introduire notre nouvelle variable d'état:

$$\frac{dADP_c}{dt} = -\gamma J_{ANT} + \frac{d}{dt} J_{ADP,ext} \quad (10)$$

on remarquera que nous avons également un nouveau flux au cours du temps, qui simule l'échange d'ADP extérieur: ce seront nos ajouts expérimentaux.

On peut noter l'ajout ponctuel d'ADP dans le milieu extérieur, via une somme. On note $[ADP]_{c,i}^+$ et t_i^+ la concentration d'ADP et le temps du début du i-ème ajout d'ADP de l'expérience. On modélise les ajouts comme ceci:

$$J_{ADP,ext} = [ADP]_c(t=0) + \sum_i [ADP]_{c,i}^+ (1 - \exp(-\frac{t-t_i^+}{\tau^+})) 1_{t>t_i^+}(t) \quad (11)$$

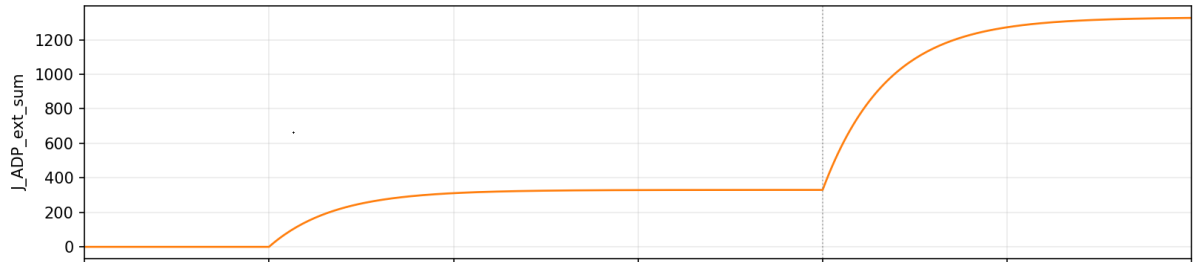


Figure 4: $J_{ANT,ext}$ au cours du temps sur $t = [14 \text{ min}, 21 \text{ min}]$

Or dans notre équation nous avons $\frac{d}{dt} J_{ADP,ext}$, donc on calcule la dérivée pour obtenir: le nouveau flux à implémenter :

$$\frac{d}{dt}J_{ADP,ext} = \sum_i \frac{[ADP]_{c,i}^+}{\tau^+} \exp\left(-\frac{t-t_i^+}{\tau^+}\right) \mathbf{1}_{t>t_i^+}(t) \quad (12)$$

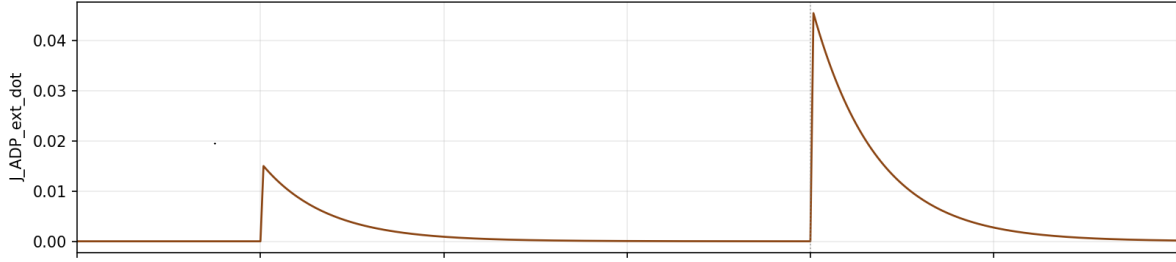


Figure 5: $\frac{d}{dt}J_{ANT,ext}$ au cours du temps sur $t=[14 \text{ min}, 21 \text{ min}]$

La vitesse d'ajout de l' ADP_c est contrôlée par le paramètre τ^+ du modèle. Dans les paramètres initiaux, nous avons $\tau^+ = 0.05$ minute. On peut ainsi observer l'évolution de $\frac{d}{dt}J_{ANT,ext}$.

De plus, avec ce nouvel apport, on doit aussi prendre en compte le changement de rapport d'ADP vis-à-vis de l'ATP. Avec cet ajout, l'hypothèse du modèle de Bertram actuel $\frac{ADP_c}{ATP_c} = \frac{ADP_m}{ATP_m}$ ne tient plus; on doit modifier également notre flux d'équilibre de J_{ANT} .

$$J_{ANT} = p_{19} \left(\frac{RAT_m}{RAT_m + p_{20}} \right) \exp(0.5FRT\Delta\Psi) \quad (13)$$

RAT_m pour expliquer un flux extérieur devient incohérent; on remplace donc par la version actuelle de J_{ANT} par la version originelle de Magnus-Keizer:

$$J_{ANT_2} = V_{maxANT} \frac{RAT_m - 0.8RAT_c}{(1 + 0.11RAT_c)(RAT_m + 7.2)e^{-0.5FRT\Delta\Psi}} \quad (14)$$

On complexifie l'étude et la calibration future en ajoutant 4 données paramétrables, V_{maxANT} 0.8 , 0.11 et 7.2

5.2 Modification et extension de l'implémentation

Nous avons opté pour l'utilisation de l'héritage en définissant une classe `BertramEtendue` dérivée de la classe `Bertram`. Cette approche permet de conserver l'ensemble des caractéristiques du modèle initial, tout en surchargeant les flux nécessaires et en ajoutant les nouveaux flux spécifiques à cette version.

Par ailleurs, en vue de la calibration future du modèle, nous avons repensé la structure globale du code. Les principales modifications apportées sont les suivantes :

- Suppression de l'utilisation de structures de type `map` associées à des fichiers `.txt` pour la gestion des paramètres du modèle ;
- Passage de l'ensemble des paramètres du modèle en arguments du programme (via `argv`) ;
- Interfaçage du code C++ avec Python, en utilisant `subprocess.run`, permettant de lancer l'exécutable C++ depuis Python avec une liste d'arguments.

5.3 Normalisation de l'oxygène

Lors des premières exécutions du programme, les résultats obtenus étaient incohérents. En effet, un jeu de données expérimental nous a été fourni, comprenant cinq séries de mesures correspondant à cinq individus distincts, avec des durées d'observation différentes. Ces données décrivent l'évolution de la consommation d'oxygène au cours du temps, qui constitue l'une des sorties attendues du modèle.

Afin de comparer les résultats de manière pertinente, toutes les sorties graphiques ont été extraites sur l'intervalle

$$t \in [14 \text{ min}, 21 \text{ min}]$$

avec un pas de temps fixé à

$$\Delta t = 1 \text{ ms.}$$

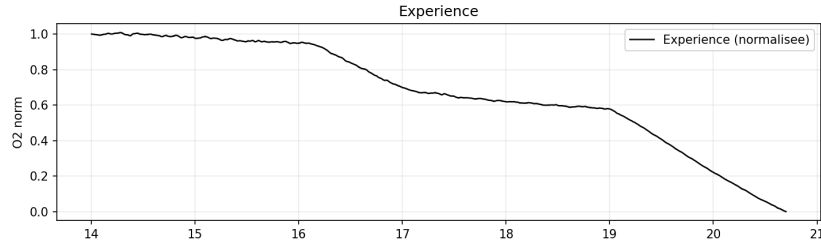


Figure 6: Visualisation des résultats expérimentaux obtenus et affichage de la normalisation de la consommation d'oxygène au cours du temps

Lors des ajouts d' ADP_c , le flux présente bien une augmentation progressive, sans perte instantanée d' ADP_c au moment de l'injection.

Nous pouvons alors nous intéresser au comportement de la variable d'état. Cette étape est centrale, puisqu'elle permet de vérifier si la dynamique imposée par les ajouts d' ADP_c se répercute correctement sur la sortie principale du modèle, soit la norme de la consommation d'oxygène.

Il devient alors possible de comparer directement les deux courbes, comme illustré ci-dessous :

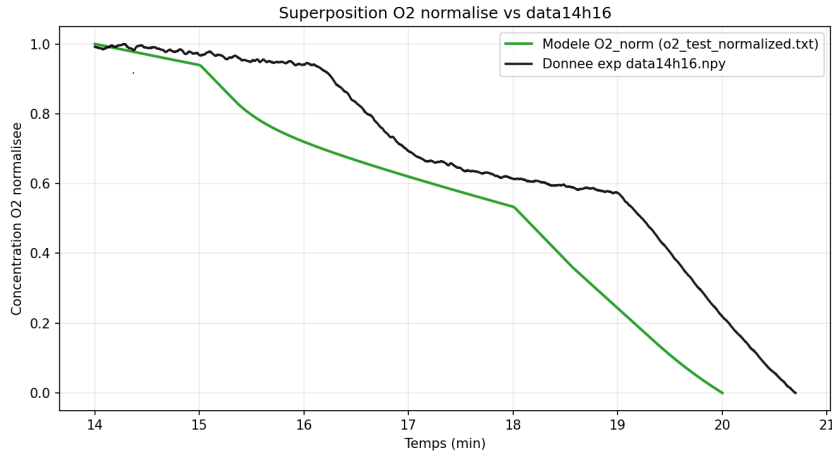


Figure 7: Validation de l'implémentation C++ vis à vis de l'expérience

6 Calibration du modèle

Notre implémentation en C++ étant désormais opérationnelle, nous pouvons passer à l'étape de calibration du modèle. Pour cela, nous allons nous appuyer sur les données expérimentales disponibles et réaliser une étude graphique des pentes observées sur les différentes courbes.

Dans cette optique, nous effectuerons des régressions linéaires sur certaines portions des signaux afin d'en extraire des rapports de pentes. Ces quantités constitueront des indicateurs centraux dans le processus de calibration.

L'objectif est alors de retrouver, à partir des courbes expérimentales, les paramètres propres à chaque individu, c'est-à-dire les données caractéristiques associées à chacun d'eux.

6.1 Régression linéaire

On effectue des régressions linéaires sur les 5 jeux de données, on notera P_1 à P_4 les pentes respectives. Exemple sur la première donnée:

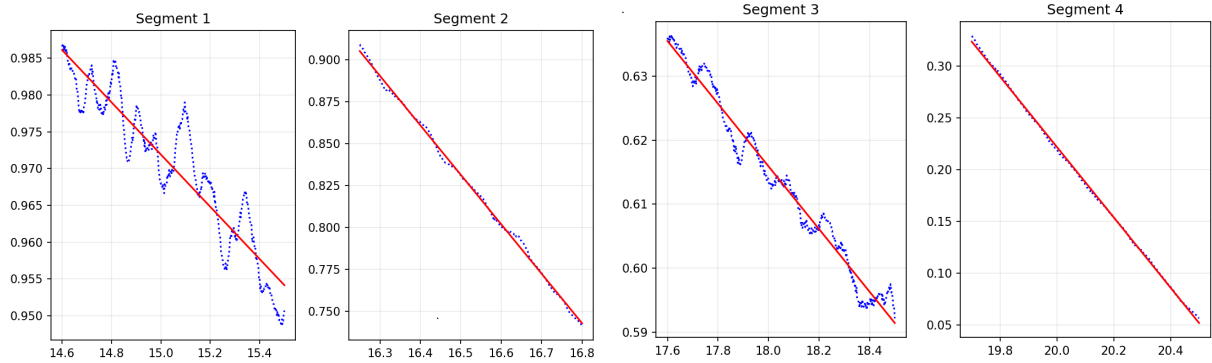


Figure 8: Régression linéaire de différents segments

Sur la courbe entière cela donne les portions suivantes sélectionnées :

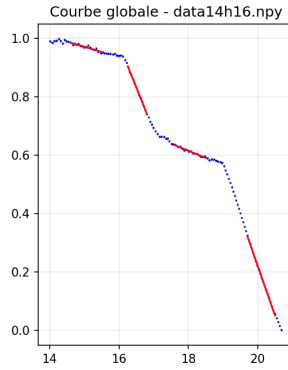


Figure 9: Régression linéaire du jeu de données 14h16.npy

Les régressions linéaires effectuées sur différentes portions des résultats expérimentaux permettent d'estimer les pentes caractéristiques. Ces pentes traduisent localement la vitesse de variation de la consommation d'oxygène, et fournissent ainsi une information plus pertinente qu'une simple comparaison visuelle des courbes complètes.

Dans notre étude, les différentes portions sélectionnées correspondent à des phases distinctes du signal: avant les ajouts d' ADP_c , pendant la réponse aux ajouts, et après ces derniers. Les pentes obtenues, notées P_1 , P_2 , P_3 et P_4 , constituent alors des indicateurs expérimentaux.

Afin de rendre la comparaison avant ajouts et après sur les courbes, nous choisissons de travailler avec les pentes elles-mêmes et également avec certains rapports de pentes.

Ces rapports permettent de quantifier l'effet des ajouts d' ADP_c sur la consommation d'oxygène. Ils constituent ainsi des paramètres pour la calibration, car ils reflètent directement la réponse du système aux perturbations expérimentales.

6.2 Construction de la calibration

L'objectif de la calibration est d'ajuster les paramètres du modèle de manière à reproduire au mieux les caractéristiques observées dans les données expérimentales. Pour cela, on compare les indicateurs extraits des données expérimentales à ceux obtenus à partir des simulations du modèle.

Plus précisément, pour un jeu de paramètres donné, le modèle produit une courbe simulée de consommation d'oxygène. Sur cette courbe, nous effectuons le même processus que pour les données

expérimentales : sélection de segments, régressions linéaires, puis extraction des pentes caractéristiques et de leurs rapports.

Nous définissons alors une fonction qui mesure l'écart entre les indicateurs expérimentaux et les indicateurs simulés. On obtient donc la somme des écarts quadratiques:

$$J(X) = \sum_{k=1}^4 (P_k^{sim}(X) - P_k^{exp})^2 + \sum_r (R_r^{sim}(X) - R_r^{exp})^2 \quad (15)$$

où :

- X désigne le vecteur des paramètres à calibrer ;
- P_k^{exp} et $P_k^{sim}(X)$ désignent respectivement les pentes expérimentales et simulées ;
- R_r^{exp} et $R_r^{sim}(X)$ désignent les rapports de pentes expérimentaux et simulés.

L'objectif de la calibration est alors de trouver un vecteur de paramètres X minimisant cette fonction.

6.3 Principe de la calibration

Le processus de calibration repose donc sur une boucle itérative. À partir d'un jeu initial de paramètres, le modèle est simulé numériquement, puis les mêmes indicateurs que sur les données expérimentales sont extraits.

Les paramètres sont ensuite ajustés afin de réduire l'écart entre la réponse simulée et la réponse expérimentale. Ce travail constitue la base de la procédure d'optimisation qui sera mise en œuvre par notre algorithme génétique.

Voici les sorties du test de notre algorithme sur une fonction simple à 4 paramètres

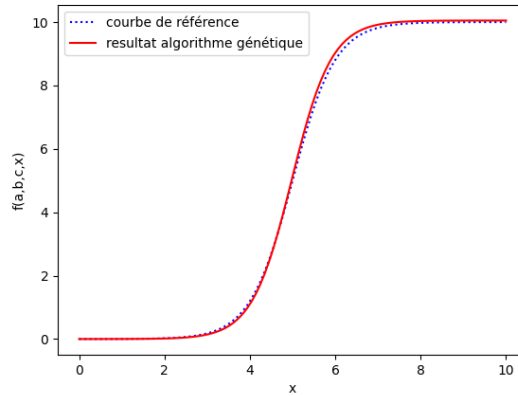


Figure 10: Test de l'algorithme génétique sur une fonction à 4 paramètres

6.4 Algorithme génétique

Nous avons élaboré un algorithme génétique, son fonctionnement est le suivant:

Dans un premier temps, on génère une population de 100 individus qui possèdent tous une marge de mutation de plus ou moins 30% des valeurs initiales référencées dans le modèle de Bertram.

Par la suite, sur ces 100 individus générés, on applique notre solveur afin d'obtenir des résultats normalisés de consommations d'oxygène; sur ces résultats, on extrait des régressions linéaires sur les mêmes intervalles que les données expérimentales.

On effectue une comparaison à l'aide de la formule de quadrature vue ci-dessus; écart entre les pentes, et écarts entre les ratios de pentes (après plusieurs essais on a modifié pour y inclure aussi la comparaison

point par point afin de réduire l'écart pour un fit optimal, on y a mis un poids faible $poids = 0,2$ pour ne pas impacter de trop notre modèle).

Les résultats de cette quadrature, on en extrait les 25 meilleurs individus , soit ceux qui ont obtenu le fit le plus bas (le plus proche de 0).

Afin de reconstruire les "enfants" de la prochaine génération, on applique une loi normale centrée au meilleur fit, et on prend un écart-type de 3% pour nos nouveaux individus générés par combinaison linéaire des 25 parents.

On réitère le processus sur 25 générations afin d'avoir une mutation contrôlée et stable.

Lors de nos premiers résultats, on a observé une stagnation du meilleur fit. Pour pallier à ceci, on a opté pour un élargissement du domaine de mutation s'il y avait stagnation sur plus de 3 générations.

Ainsi, nous avons pu obtenir les résultats finaux suivants: (relatifs aux données expérimentales)

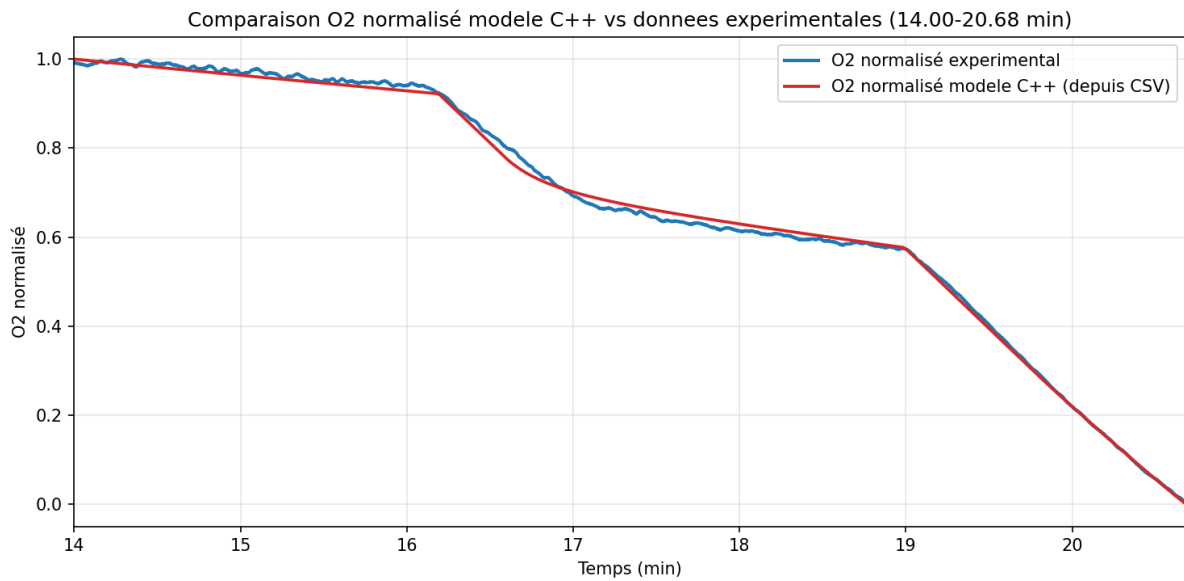


Figure 11: Visualisation de la calibration des données expérimentales avec du bruit (en bleu) et des paramètres obtenus par l'algorithme génétique (en rouge)

Voici les paramètres obtenus de ce fit, que l'on peut comparer directement avec les paramètres initiaux de Bertram :

Modèle	p1	p2	p3	p4	p5	p6	p7	p8	p9	p10
Bertram paramètres	400	1	0.01	0.6	100	177	5	7	100	177
Algorithme génétique	422.552	0.956	0.010	0.552	105.857	181.296	4.928	6.125	109.547	188.133

Modèle	p11	p12	p13	p14	p15	p16	p17	p18	p19	p20
Bertram paramètre	5	120	10000	190	8.5	35	0.002	-0.03	0.35	2
Algorithme génétiques	5.443	127.414	8885.396	182.809	9.053	27.964	0.002	-0.033	0.396	2.040

Modèle	p21	p22	p23	p24	FRT	NAD_{tot}	A_{tot}	C_m	f_m	k_{GPDH}
Bertram paramètres	0.01	1.1	0.001	0.016	0.037	10000	15000	1.8	0.01	0.0005
Algorithme génétique	0.009	0.949	0.0009	0.017	0.036	9266.094	13921.33	1.690	0.008	0.0004

De ces résultats, avec une comparaison d'erreur cumulée, on a une erreur minime avec un fit à 1.92, ce qui est totalement tolérable comme approximation compte tenu du bruit de l'expérience qui perturbe les résultats

Nous avons fait ceci pour les 4 jeux de données restants :

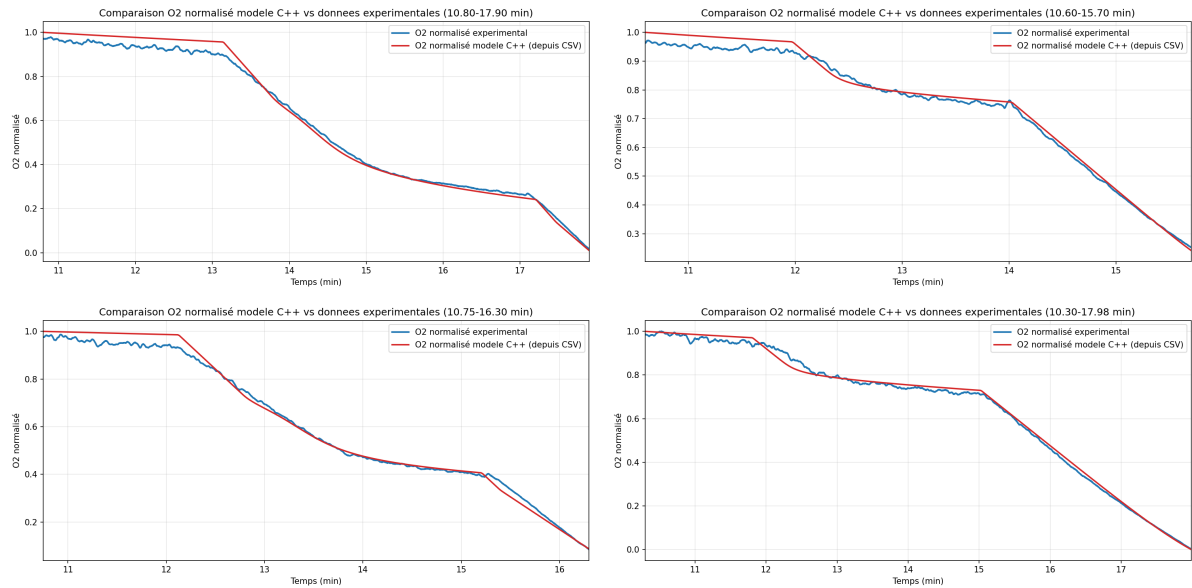


Figure 12: Visualisation des paramètres obtenus de la consommation d'oxygène vis-à-vis de leurs courbes expérimentales

7 Perspectives et Conclusion

Ce projet collaboratif encadré nous a permis d'explorer un modèle mathématique de la réponse mitochondriale, d'en comprendre les mécanismes principaux, puis de le faire évoluer afin de l'adapter à une problématique expérimentale spécifique : l'étude de la respiration mitochondriale.

Au cours de ce travail, nous avons été confrontés à des enjeux à la fois biologiques, mathématiques et numériques. L'analyse du modèle et son implémentation ont mis en évidence la difficulté de représenter fidèlement des phénomènes complexes tout en conservant un cadre exploitable numériquement. La phase de calibration, en particulier, a souligné l'importance du choix des méthodes d'optimisation, mais aussi la contrainte majeure que représente le temps de calcul, notamment dans un contexte où un grand nombre de paramètres doivent être ajustés simultanément.

Les résultats obtenus constituent une première base encourageante, mais plusieurs améliorations pourraient être envisagées. Parmi elles, l'ajustement des paramètres de l'algorithme génétique (taille de population, nombre de générations, taux de mutation), la réduction de l'espace de recherche en fixant certains paramètres peu influents, ou encore l'utilisation de solutions intermédiaires comme points de départ pour d'autres calibrations pourraient permettre d'améliorer à la fois l'efficacité et la robustesse de la méthode.

Enfin, ce projet a montré qu'en modélisation comme en optimisation, de légères modifications dans les hypothèses ou dans les choix algorithmiques peuvent produire des effets significatifs sur les résultats. Cette sensibilité souligne l'importance d'une réflexion rigoureuse sur les outils utilisés et sur leur adéquation avec la problématique étudiée.

Ainsi, au-delà de ses résultats immédiats, ce travail nous a permis de mieux appréhender la démarche scientifique dans sa globalité : comprendre un phénomène, le traduire mathématiquement, le simuler numériquement, puis confronter les résultats obtenus aux données expérimentales. Il ouvre également des perspectives intéressantes pour la poursuite de travaux en modélisation biologique et en calibration de systèmes dynamiques complexes.

References

- [1] Richard Bertram et al. “A simplified model for mitochondrial ATP production”. In: *Journal of Theoretical Biology* 243.4 (2006), pp. 575–586. URL: <https://www.math.fsu.edu/~bertram/papers/beta/simpleMK.pdf>.
- [2] CellML Team. *Bertram et al. 2006 Mitochondrial ATP Production Model*. Modèle CellML consulté en ligne. 2006. URL: https://models.cellml.org/exposure/a7f8b4574fe4802f1e06f247006962ac/bertram_2006.cellml/@cellml_codegen/Python.